Module 5：視覺化與 EDA

課程定位

所屬課程：2026 Python 數據分析與 AI 基礎：從資料思維到機器學習入門（24 小時）
模組編號：Module 5／共 8 個模組 時長：3 小時 前置模組：M4 pandas 與資料清理 後續模組：M6 統計思維與假設檢定

模組核心主張：圖表是思考工具，不是裝飾品。EDA 的終點不是一張漂亮圖，而是一個能被驗證的業務假設。

> "The greatest value of a picture is when it forces us to notice what we never expected to see." — John Tukey

---

學習目標

完成本模組後，學員能夠：

1. 說明 Matplotlib 與 Seaborn 的定位差異，並在正確場景選用適當套件
2. 使用 Matplotlib 基本語法建立常見圖形，包含自訂標題、軸標籤、色彩與圖例
3. 使用 Seaborn 高階 API 快速產出統計圖形，包含 histogram、boxplot、scatterplot、heatmap
4. 說明四種圖表任務類型（分佈、比較、關聯、時序），並為給定問題選擇正確圖形
5. 依照 EDA 標準路徑（單變數 → 雙變數 → 多變數）系統性探索一份陌生資料集
6. 從 EDA 結果中提取洞察，並以「觀察 → 洞察 → 建議」結構輸出業務故事
7. 識別資料中的異常值與分佈特徵，判斷其是資料品質問題還是業務訊號
8. 獨立完成一份包含三個洞察的 EDA 報告

關鍵概念清單

視覺化工具

- [ ] Matplotlib 基礎架構：Figure、Axes、Artist 三層模型
- [ ] plt.subplots() 建立單圖與多子圖
- [ ] 常用圖形 API：plt.plot()、plt.bar()、plt.hist()、plt.scatter()、plt.boxplot()
- [ ] 圖形客製化：title、xlabel、ylabel、legend、color、figsize
- [ ] Seaborn 與 Matplotlib 的關係：Seaborn 建立在 Matplotlib 之上
- [ ] Seaborn 主要圖形：sns.histplot()、sns.boxplot()、sns.scatterplot()、sns.heatmap()、sns.pairplot()
- [ ] 探索圖 vs 報告圖的用途差異

EDA 方法論

- [ ] 圖表任務四分類：分佈圖、比較圖、關聯圖、時序圖
- [ ] EDA 三層路徑：單變數分析、雙變數分析、多變數分析
- [ ] 好的資料問題三要素：可觀察、可比較、可驗證
- [ ] 異常值識別：IQR 法則、3σ 法則、視覺化判讀
- [ ] 相關性矩陣：Pearson 相關係數的直覺意義與局限
- [ ] 資料故事結構：觀察（Observation）→ 洞察（Insight）→ 建議（Recommendation）

投影片大綱

#	投影片標題	核心訊息
1	圖表不是裝飾，是思考的外掛	圖表的價值在揭露，不在呈現

2	Matplotlib：視覺化的基礎層	先懂基礎，才能駕馭高階工具
3	Seaborn：統計視覺化的快車道	一行程式碼做出正確的統計圖
4	四種圖表任務類型	選圖從問題出發，不從套件出發
5	什麼是好的資料問題？	問題的品質決定圖的品質
6	EDA 路徑：單變數分析	先看每個變數的形狀，再問關係
7	EDA 路徑：雙變數與多變數分析	關係藏在變數之間，不在變數之內
8	異常值：資料品質問題還是業務訊號	異常值不等於錯誤，先判斷再處理
9	真實案例：電商用戶行為 EDA 全流程	看一次完整的 EDA 怎麼進行
10	如何用一頁圖表講業務故事	從觀察到洞察到建議的轉化
11	【練習 A】完整 EDA 探索	實作：走完完整的 EDA 路徑
12	【練習 B】三段洞察簡報	實作：一份資料集，三個洞察

詳細投影片內容

Slide 1：圖表不是裝飾，是思考的外掛

核心訊息：圖表不是用來讓報告看起來專業，而是用來幫你發現資料裡肉眼看不到的結構。

講師講解要點：

- 大多數人學視覺化的動機是「讓報告好看」，但這是本末倒置——好圖的標準是「它讓你發現了什麼」
- 試著用純數字表格回答「這兩組的差異大嗎？」，再換成圖，感受資訊密度的差異
- Anscombe's Quartet：四組統計數字完全相同的資料集（均值、變異數、相關係數全部一樣），但一畫出來結構完全不同
- EDA 中的圖是給自己看的草稿，不是給老闆看的成品——探索圖和報告圖是兩件事，要分開對待
- 重要觀念：探索階段的圖目的是「問問題」，報告階段的圖目的是「傳達答案」

程式碼範例：

```
import matplotlib.pyplot as plt
import numpy as np

# Anscombe's Quartet - [redacted]
datasets = {
    'I':   {'x': [10,8,13,9,11,14,6,4,12,7,5],  'y': [8.04,6.95,7.58,8.81,8.33,9.96,7.24,4.26,10.84,
    'II':  {'x': [10,8,13,9,11,14,6,4,12,7,5],  'y': [9.14,8.14,8.74,8.77,9.26,8.10,6.13,3.10,9.13,
    'III': {'x': [10,8,13,9,11,14,6,4,12,7,5],  'y': [7.46,6.77,12.74,7.11,7.81,8.84,6.08,5.39,8.15,
    'IV':  {'x': [8,8,8,8,8,8,8,19,8,8,8],      'y': [6.58,5.76,7.71,8.84,8.47,7.04,5.25,12.50,5.56,
}

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, (name, data) in zip(axes, datasets.items()):
    ax.scatter(data['x'], data['y'], color='steelblue', alpha=0.8, s=60)
    ax.set_title(f'Dataset {name}', fontsize=13)
    ax.set_xlabel('x'); ax.set_ylabel('y')
    ax.set_xlim(0, 20); ax.set_ylim(0, 14)

fig.suptitle('Anscombe's Quartet[redacted]', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

視覺建議：左右對比圖——左側是一張密密麻麻的數字表格，右側是 Anscombe's Quartet 的四個散佈圖。底部標注：「這四組資料的均值、變異數、相關係數完全相同——你還覺得數字夠用嗎？」

銜接下一張：「既然視覺化這麼重要，我們先從最基礎的工具 Matplotlib 開始，建立正確的操作模型。」

Slide 2：Matplotlib：視覺化的基礎層

核心訊息：Matplotlib 是 Python 視覺化的基礎。理解它的三層架構，你才能精確控制每一個視覺細節。

講師講解要點：

- 三層架構：
- Figure：整張畫布（canvas），可以包含一個或多個子圖
- Axes：一個獨立的座標系（subplot），是實際畫圖的地方
- Artist：所有可見元素（線條、點、文字、刻度）都是 Artist 物件
- 兩種使用風格：
- `plt.xxx()` 直接呼叫（快速但不靈活，適合單圖快速探索）
- `fig, ax = plt.subplots()` 物件導向（明確、可擴展，推薦用於正式圖表）
- 常見圖形 API

  速查：`ax.plot()`（折線）、`ax.bar()`（長條）、`ax.hist()`（直方圖）、`ax.scatter()`（散佈）、`ax.boxplo`
- 圖形客製化核心參數：`title`、`xlabel/ylabel`、`legend`、`color`、`figsize`、`alpha`

程式碼範例：

```
import matplotlib.pyplot as plt
import numpy as np

# ---- [redacted] pyplot [redacted] ----
x = np.linspace(0, 10, 100)
plt.figure(figsize=(8, 4))
plt.plot(x, np.sin(x), label='sin(x)', color='steelblue', linewidth=2)
plt.plot(x, np.cos(x), label='cos(x)', color='coral', linewidth=2, linestyle='--')
plt.title('[redacted]', fontsize=14)
plt.xlabel('x [redacted]'); plt.ylabel('y [redacted]')
plt.legend(); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

# ---- [redacted] ----
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# [redacted]
categories = ['A [redacted]', 'B [redacted]', 'C [redacted]', 'D [redacted]']
values     = [23, 45, 31, 52]
axes[0].bar(categories, values, color=['#4C72B0','#DD8452','#55A868','#C44E52'], alpha=0.85)
axes[0].set_title('[redacted]'); axes[0].set_ylabel('[redacted]')

# [redacted]
data = np.random.normal(loc=170, scale=8, size=500)
axes[1].hist(data, bins=25, color='steelblue', alpha=0.75, edgecolor='white')
```

```
axes[1].set_title('[redacted]'); axes[1].set_xlabel('[redacted] (cm)')
[redacted]
# [redacted]
x_s = np.random.normal(0, 1, 200)
y_s = 2 * x_s + np.random.normal(0, 0.5, 200)
axes[2].scatter(x_s, y_s, alpha=0.5, color='coral', s=30)
axes[2].set_title('[redacted]'); axes[2].set_xlabel('X'); axes[2].set_ylabel('Y')
[redacted]
plt.suptitle('Matplotlib [redacted]', fontsize=14, y=1.02)
plt.tight_layout(); plt.show()
```

視覺建議：上方顯示 Figure / Axes / Artist 三層架構示意圖（巢狀方框），下方顯示程式碼範例產生的三種圖形。每個圖形旁邊標注適用情境關鍵字。

銜接下一張：「Matplotlib 靈活但需要寫不少程式碼。當我們想快速產出統計圖時，Seaborn 大幅降低了門檻。」

Slide 3：Seaborn：統計視覺化的快車道

核心訊息：Seaborn 建立在 Matplotlib 之上，一行程式碼就能畫出正確的統計圖形，並自動處理好配色與格式。

講師講解要點：

- Seaborn vs Matplotlib 的定位差異：
- Matplotlib：底層控制，適合客製化複雜圖形
- Seaborn：高階封裝，適合快速統計探索
- 兩者並不互斥——Seaborn 畫完之後，可以用 Matplotlib API 進一步微調
- Seaborn 常用圖形速查：
- `sns.histplot(data, x='col')` — 帶 KDE 的直方圖
- `sns.boxplot(data, x='group', y='value')` — 分組箱形圖
- `sns.scatterplot(data, x='col1', y='col2', hue='category')` — 帶類別色彩的散佈圖
- `sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')` — 相關性熱力圖
- `sns.pairplot(data, hue='category')` — 多變數配對圖（EDA 神器）
- 主題設定：`sns.set_theme(style='whitegrid')` 一行讓所有圖形更好看

程式碼範例：

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme(style='whitegrid', palette='muted')

# [redacted] seaborn [redacted]
tips = sns.load_dataset('tips')

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 1. [redacted] + KDE[redacted]
sns.histplot(data=tips, x='total_bill', kde=True, ax=axes[0,0], color='steelblue')
axes[0,0].set_title('[redacted] KDE')

# 2. [redacted]
sns.boxplot(data=tips, x='day', y='total_bill', hue='sex', ax=axes[0,1])
axes[0,1].set_title('[redacted]')

# 3. [redacted] vs [redacted]
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='smoker',
                style='time', alpha=0.8, ax=axes[1,0])
axes[1,0].set_title('[redacted] vs [redacted]')

# 4. [redacted]
corr = tips[['total_bill', 'tip', 'size']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1,
            fmt='.2f', ax=axes[1,1])
axes[1,1].set_title('[redacted]')

plt.suptitle('Seaborn [redacted]', fontsize=15, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：2x2 子圖格，每個圖形下方標注對應的 Seaborn API 呼叫（一行程式碼）。旁邊加一個對比表格：同一張箱形圖，左側 Matplotlib 版本需要 12 行，右側 Seaborn 版本只需要 1 行。

銜接下一張：
「工具就位了。接下來的關鍵問題是：面對一份資料，我應該畫什麼圖？答案是：先問問題，問題決定圖形。」

Slide 4：四種圖表任務類型

核心訊息：選圖的起點是問題，不是套件。先問「我想知道什麼」，再選工具。

講師講解要點：

- 分佈圖（Distribution）：「這筆數值的形狀是什麼？集中還是分散？有沒有雙峰？」
- 工具：histogram、KDE plot、boxplot、violin plot

範例問題：「用戶停留時間的分佈是正態的嗎？有沒有重尾？」

比較圖（Comparison）：「A 和 B 哪個比較大？不同群組之間有差異嗎？」

- 工具：bar chart、grouped boxplot、dot plot

範例問題：「週末的平均訂單金額比平日高嗎？」

關聯圖（Relationship）：「X 變動時，Y 怎麼變？有沒有相關性或群集？」

- 工具：scatter plot、heatmap、pairplot、bubble chart

範例問題：「廣告投放金額和轉換率之間有什麼關係？」

時序圖（Time Series）：「這個指標隨時間如何演變？有沒有趨勢或季節性？」

- 工具：line chart、area chart、candlestick

範例問題：「過去 12 個月的月活躍用戶數趨勢如何？」

常見錯誤示範：想問「A 和 B 哪個比較大」卻畫了 pie chart；想看分佈卻只算均值

程式碼範例：

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

sns.set_theme(style='whitegrid')
np.random.seed(42)
n = 300

df = pd.DataFrame(
    'purchase_amount': np.concatenate(
        np.random.normal(200, 50, 200),   # 一般客
        np.random.normal(800, 100, 100)   # 大客戶
    ),
    'channel': np.random.choice(['organic', 'paid', 'social'], n),
    'ad_spend': np.random.uniform(100, 5000, n),
    'conversion_rate': None
```

```
    'date': pd.date_range('2024-01-01', periods=n, freq='D')[:n],
})
df['conversion_rate'] = 0.05 + df['ad_spend'] / 100000 + np.random.normal(0, 0.01, n)

fig, axes = plt.subplots(2, 2, figsize=(13, 10))

# [redacted]
sns.histplot(df['purchase_amount'], kde=True, bins=30, ax=axes[0,0], color='steelblue')
axes[0,0].set_title('[redacted]')
axes[0,0].set_xlabel('[redacted]')

# [redacted]
sns.boxplot(data=df, x='channel', y='purchase_amount', ax=axes[0,1], palette='pastel')
axes[0,1].set_title('[redacted]')

# [redacted]
sns.scatterplot(data=df, x='ad_spend', y='conversion_rate',
                hue='channel', alpha=0.6, ax=axes[1,0])
axes[1,0].set_title('[redacted] vs [redacted]')

# [redacted]
monthly = df.set_index('date')['purchase_amount'].resample('W').mean()
axes[1,1].plot(monthly.index, monthly.values, color='coral', linewidth=2)
axes[1,1].set_title('[redacted]')
axes[1,1].set_xlabel('[redacted]'); axes[1,1].tick_params(axis='x', rotation=30)

plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：2x2 卡片格，每格一個任務類型，包含：任務名稱、代表問題範例、縮圖示意、適合的圖形工具列表。使用一致的色

銜接下一張：「知道有哪些圖，但還不夠——更根本的問題是：什麼樣的資料問題值得畫圖？」

Slide 5：什麼是好的資料問題？

核心訊息：圖來自問題，不是來自資料集。沒有問題，圖只是雜訊。

講師講解要點：

- 壞問題範例：「我想看看這份資料長什麼樣。」→ 沒有焦點，會畫出 20 張沒有結論的圖
- 好問題的三個條件：
- 可觀察：能在資料裡找到對應的欄位
- 可比較：有比較基準（群組、時間、目標值）

- 可驗證：結果能幫助做決定或推進分析
- 好問題範例：「新用戶的首週購買金額，是否比老用戶低？」
- 可觀察：有購買金額欄位✓
- 可比較：新用戶 vs 老用戶✓
- 可驗證：結果影響行銷策略✓
- 問題從哪裡來？業務痛點、異常警示、目標對齊——不是「看到什麼欄位就問什麼問題」
- 練習思維轉換：給定一個業務情境，先寫下問題，再決定畫哪種圖

壞問題 vs 好問題對照表：

壞問題	問題出在哪裡	改良後的好問題
「分析一下銷售資料」	沒有方向，無法畫圖	「哪個商品類別的退貨率最高？」
「看看用戶行為」	沒有比較基準	「新用戶和回訪用戶的瀏覽深度有差異嗎？」
「了解廣告效果」	沒有具體指標	「投放 FB 廣告的 ROI 是否優於 Google 廣告？」

視覺建議：
三欄對比表：左欄「壞問題」（模糊、無方向），中欄「好問題三要素」（可觀察、可比較、可驗證），右欄「
→ 問題 → 圖表任務類型 → 選圖工具。

銜接下一張：「有了問題，怎麼系統性地探索一份新資料？從單一變數開始。」

Slide 6：EDA 路徑：單變數分析

核心訊息：EDA
的第一步是看每個變數的形狀——不是關係，不是趨勢，是每個欄位本身的分佈與特性。

講師講解要點：

- 為什麼從單變數開始？

- 在研究兩個變數的關係之前，先確認每個變數「正不正常」
- 單變數分析能發現：異常值、偏態、資料品質問題、意外的群集
- 數值型變數的單變數分析：
- `df.describe()` 快速摘要（但記住：均值和標準差不能代表全部）
- Histogram 看形狀：對稱？偏態？雙峰？
- Boxplot 找異常：四分位距（IQR）法則識別可疑值
- 計算偏度（skewness）：`df['col'].skew()` > 1 代表右偏
- 類別型變數的單變數分析：
- `df['col'].value_counts()` 頻率分佈
- Bar chart 視覺化類別分佈
- 注意：是否有「其他」類別佔大多數？類別數量是否合理？

程式碼範例：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style='whitegrid')
np.random.seed(42)

# ========
n = 1000
df = pd.DataFrame(
    'order_amount':   np.concatenate([np.random.lognormal(5, 0.8, 950),
                                      np.random.uniform(5000, 20000, 50)]),  # ====
    'category':       np.random.choice(['==', '==', '==', '==', '=='],
                                       n, p=[0.25, 0.30, 0.20, 0.15, 0.10]),
    'is_new_user':    np.random.choice([True, False], n, p=[0.35, 0.65]),
    'days_since_join': np.random.exponential(180, n).astype(int)
)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# ========
sns.histplot(df['order_amount'], kde=True, bins=50, ax=axes[0,0], color='steelblue')
axes[0,0].set_title('==========');  axes[0,0].set_xlabel('==')

# ==========
sns.boxplot(y=df['order_amount'], ax=axes[0,1], color='lightcoral')
axes[0,1].set_title('==========
```

```
[redacted]
[redacted]
sns.histplot(np.log1p(df['order_amount']), kde=True, bins=40, ax=axes[0,2], color='mediumseagreen
axes[0,2].set_title('log([redacted]) [redacted]')
[redacted]
[redacted]
cat_counts = df['category'].value_counts()
axes[1,0].barh(cat_counts.index, cat_counts.values, color=sns.color_palette('pastel'))
axes[1,0].set_title('[redacted]'); axes[1,0].set_ylabel('[redacted]')
[redacted]
[redacted]
sns.histplot(df['days_since_join'], bins=40, kde=True, ax=axes[1,1], color='plum')
axes[1,1].set_title('[redacted]')
[redacted]
[redacted]
print("=== [redacted] ===")
print(df['order_amount'].describe().round(2))
print(f"\n[redacted] (Skewness): {df['order_amount'].skew():.2f}")
print(f"[redacted] (Kurtosis): {df['order_amount'].kurt():.2f}")
[redacted]
axes[1,2].axis('off')
plt.suptitle('[redacted] + [redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議： 2x3 子圖格，同一個數值欄位用三種不同圖形展示（histogram、boxplot、log 轉換後），讓學員看到「同一份資料，不同圖形揭露不同問題」。

銜接下一張：「看完每個變數後，接下來最關鍵的問題出現了：變數之間有什麼關係？」

Slide 7：EDA 路徑：雙變數與多變數分析

核心訊息：洞察藏在變數的關係裡。雙變數分析看兩個欄位的互動；多變數分析在複雜關係中找結構。

講師講解要點：

- 雙變數分析類型：
- 數值 vs 數值：Scatter plot + 相關係數
- 數值 vs 類別：Grouped boxplot / violin plot
- 類別 vs 類別：Crosstab + heatmap / grouped bar chart
- 相關係數的直覺：
- Pearson r 的範圍：-1（完全負相關）到 +1（完全正相關）
- |r| > 0.7 → 強相關；0.3-0.7 → 中度相關；< 0.3 → 弱相關

- 陷阱：相關性 ≠ 因果關係；非線性關係 r 也可能很低
- 多變數分析：
- `sns.pairplot()` 一次看所有數值欄位的兩兩關係（EDA 神器）
- Correlation heatmap：快速找出哪些欄位強相關
- 用 `hue` 參數加入類別維度，看群組在多維空間的分佈

程式碼範例：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_theme(style='whitegrid')
np.random.seed(42)

# 模擬資料
n = 400
df = pd.DataFrame({
    'order_amount':    np.random.lognormal(5, 0.7, n),
    'browsing_time':   np.random.exponential(15, n),
    'num_items':       np.random.poisson(3, n) + 1,
    'discount_rate':   np.random.uniform(0, 0.5, n),
    'user_type':       np.random.choice(['新客', '回客', 'VIP'], n, p=[0.3, 0.5, 0.2])
})
# 加入相關性：num_items 越多，order_amount 越高
df['order_amount'] = df['order_amount'] * (1 + df['num_items'] * 0.1)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# 散點圖：數量 vs 金額，按顧客類型上色
sns.scatterplot(data=df, x='num_items', y='order_amount', alpha=0.5,
                hue='user_type', ax=axes[0,0])
axes[0,0].set_title('數量 vs 金額')

# 箱型圖：各類顧客的金額分佈（grouped boxplot）
sns.boxplot(data=df, x='user_type', y='order_amount',
            order=['新客', '回客', 'VIP'],
            palette='Set2', ax=axes[0,1])
axes[0,1].set_title('顧客類型 vs 金額分佈')

# 小提琴圖：瀏覽時間 vs 顧客類型（看密度形狀）
sns.violinplot(data=df, x='user_type', y='browsing_time',
               order=['新客', '回客', 'VIP'],
               palette='pastel', ax=axes[0,2])
axes[0,2].set_title('瀏覽時間 vs 顧客類型')

# 相關性熱圖
numeric_cols = ['order_amount', 'browsing_time', 'num_items', 'discount_rate']
```

```
corr = df[numeric_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1,
            fmt='.2f', square=True, ax=axes[1,0])
axes[1,0].set_title('[redacted]')
[redacted]
axes[1,1].axis('off')
axes[1,1].text(0.1, 0.6, "[redacted]", fontsize=14, fontweight='bold', transform=axes[1,1].transAxes)
axes[1,1].text(0.1, 0.45, "[redacted] vs [redacted]r = 0.97", fontsize=10, transform=axes[1,1].transAxe
axes[1,1].text(0.1, 0.35, "[redacted]", fontsize=10, color='coral', transform=axes[1,1].transAxe
axes[1,1].text(0.1, 0.20, "[redacted] ≠ [redacted]\n[redacted]", fontsize=10, transform=axes[1,1]
[redacted]
[redacted]pairplot [redacted]
axes[1,2].axis('off')
axes[1,2].text(0.5, 0.5, "sns.pairplot(df, hue='user_type')\n\n[redacted]",
               ha='center', va='center', fontsize=11, transform=axes[1,2].transAxes,
               bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.8))
[redacted]
plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
```

視覺建議：

重點標注相關性矩陣中顏色深淺代表的含義，以及「冰淇淋與溺水」的反例插圖，強化「相關性不等於因果」的

銜接下一張：

「分析過程中，我們常常遇到讓人困惑的點：那些看起來不正常的值。它們是錯誤還是訊號？」

Slide 8：異常值：資料品質問題還是業務訊號？

核心訊息：異常值不等於錯誤。在刪除之前，先判斷它的來源——有些異常值是最有價值的洞察。

講師講解要點：

- 異常值的兩種來源：
- 資料品質問題：輸入錯誤（年齡 = 999）、系統 bug（金額 = -1）、合併錯誤
- 真實業務訊號：超級 VIP 的高消費訂單、惡意刷單行為、黑天鵝事件
- 識別方法：
- IQR 法則：Q1 - 1.5×IQR 到 Q3 + 1.5×IQR 之外視為異常（Boxplot 就是這個邏輯）
- 3σ 法則：均值 ± 3 個標準差之外（適用常態分佈）
- 視覺化判讀：Boxplot、Scatter plot、直方圖尾部

- 判斷流程：
- 識別（在哪裡？有多少筆？）
- 分類（資料品質問題 or 真實業務事件？）
- 處理（刪除 / 修正 / 保留並標記 / 單獨分析）
- 最重要的原則：永遠不要不問原因就刪除異常值

程式碼範例：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)
n = 500
amounts = np.concatenate([
    np.random.lognormal(5, 0.6, 480),      # 正常交易
    np.random.uniform(30000, 80000, 10),   # 大額 VIP 交易
    [-99, 0, 999999, 100000, -1]           # 異常值
])

# IQR 方法
Q1, Q3 = np.percentile(amounts, [25, 75])
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
outliers = amounts[(amounts < lower) | (amounts > upper)]

print(f"Q1={Q1:.0f}, Q3={Q3:.0f}, IQR={IQR:.0f}")
print(f"正常範圍: [{lower:.0f}, {upper:.0f}]")
print(f"異常值 {len(outliers)} 筆: {sorted(outliers)[:10]}")

# 畫圖
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# 直方圖
axes[0].hist(amounts, bins=50, color='steelblue', alpha=0.7)
axes[0].axvline(lower, color='red', linestyle='--', label=f'下限 {lower:.0f}')
axes[0].axvline(upper, color='red', linestyle='--', label=f'上限 {upper:.0f}')
axes[0].set_title('金額分布 + 異常值'); axes[0].legend()

# Boxplot：IQR 視覺化
axes[1].boxplot(amounts, vert=True, patch_artist=True,
                boxprops=dict(facecolor='lightblue'))
axes[1].set_title('Boxplot：四分位數')
axes[1].set_ylabel('金額')

# 清洗後
clean = amounts[(amounts > 0) & (amounts < 200000)]  # 簡單規則
axes[2].hist(clean, bins=50, color='mediumseagreen', alpha=0.7)
```

```
axes[2].set_title('[redacted]')
[redacted]
plt.suptitle('[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()
[redacted]
[redacted]
strategies = {
    '[redacted] (-99, -1)':    '[redacted] → [redacted]',
    '[redacted] (0)':          '[redacted]',
    '[redacted] VIP [redacted]':  '[redacted] → [redacted]',
    '[redacted] (999999)':  '[redacted] → [redacted]'
[redacted]
print("\n=== [redacted] ===")
for k, v in strategies.items():
    print(f"  {k}: {v}")
```

視覺建議： 左側顯示三步決策流程（識別 → 分類 → 處理），右側顯示一個具體範例：同樣是「金額 > 上界」，VIP 大單（保留）和輸入錯誤（刪除）的不同處理邏輯。

銜接下一張：「現在讓我們把前面學到的東西整合成一個完整的 EDA 實戰流程。」

Slide 9：真實案例：電商用戶行為 EDA 全流程

核心訊息：真實的 EDA 不是線性的——它是不斷從發現到追問再到發現的迭代過程。讓我們完整走一遍。

講師講解要點：

此投影片展示一個完整的 EDA 流程，從資料取得到形成假設，模擬真實工作中的思考過程。

業務情境： 電商平台希望了解：「哪些因素影響用戶的首次購買行為？如何提升新用戶轉換率？」

EDA 全流程程式碼：

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats
[redacted]
sns.set_theme(style='whitegrid', palette='muted')
np.random.seed(42)
```

```
# ========== [redacted] 0 [redacted] ==========
n = 2000
df = pd.DataFrame({
    'user_id':          range(1, n+1),
    'register_channel': np.random.choice(['organic', 'paid_search', 'social', 'referral'],
                                         n, p=[0.35, 0.30, 0.25, 0.10]),
    'device':           np.random.choice(['mobile', 'desktop', 'tablet'], n, p=[0.55, 0.35, 0.10]),
    'city_tier':        np.random.choice([1, 2, 3], n, p=[0.30, 0.45, 0.25]),
    'browsing_pages':   np.random.poisson(8, n) + 1,
    'session_duration': np.random.exponential(12, n),  # [redacted]
    'days_to_purchase': np.random.exponential(5, n)    # [redacted]
})
# [redacted]
purchase_prob = 0.05 + df['browsing_pages'] * 0.03 + (df['device'] == 'desktop').astype(float) * 0
purchase_prob = purchase_prob.clip(0, 0.85)
df['converted'] = np.random.binomial(1, purchase_prob)
df['first_order_amount'] = np.where(
    df['converted'] == 1,
    np.random.lognormal(5.2, 0.6, n),
    np.nan
)

print("=" * 50)
print("[redacted] 1[redacted]")
print("=" * 50)
print(f"[redacted]{df.shape}")
print(f"\n[redacted]\n{df.dtypes}")
print(f"\n[redacted]\n{df.isnull().sum()}")
print(f"\n[redacted]\n{df.describe().round(2)}")
print(f"\n[redacted]{df['converted'].mean():.1%}")

# ---------- [redacted] 2[redacted] ----------
print("\n" + "=" * 50)
print("[redacted] 2[redacted]")
print("=" * 50)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

# [redacted]
channel_counts = df['register_channel'].value_counts()
axes[0,0].bar(channel_counts.index, channel_counts.values,
              color=sns.color_palette('pastel'))
axes[0,0].set_title('[redacted]'); axes[0,0].set_ylabel('[redacted]')

# [redacted]
sns.histplot(df['browsing_pages'], bins=30, kde=False, ax=axes[0,1], color='steelblue')
axes[0,1].set_title(f'[redacted]={df['browsing_pages'].mean():.1f}[redacted]')

# [redacted]
sns.histplot(df['session_duration'], bins=40, kde=True, ax=axes[0,2], color='coral')
axes[0,2].set_title('[redacted]')

# [redacted]
device_counts = df['device'].value_counts()
axes[1,0].pie(device_counts.values, labels=device_counts.index,
```

```
                    autopct='%1.1f%%', colors=sns.color_palette('Set2'))
axes[1,0].set_title('[redacted]')

# [redacted]
city_counts = df['city_tier'].value_counts()
axes[1,1].bar(city_counts.index, city_counts.values, color=sns.color_palette('muted'))
axes[1,1].set_title('[redacted]')

# [redacted]
converted_df = df[df['converted'] == 1]
sns.histplot(converted_df['first_order_amount'], bins=40, kde=True, ax=axes[1,2], color='mediumseg
axes[1,2].set_title(f'[redacted]n={len(converted_df)}[redacted]')

plt.suptitle('[redacted] 2[redacted]', fontsize=14, y=1.01)
plt.tight_layout(); plt.show()

# ========== [redacted] 3[redacted]==========
print("\n" + "=" * 50)
print("[redacted] 3[redacted] — [redacted]")
print("=" * 50)

fig, axes = plt.subplots(2, 2, figsize=(13, 10))

# [redacted] vs [redacted]
channel_conv = df.groupby('register_channel')['converted'].mean().sort_values(ascending=False)
axes[0,0].bar(channel_conv.index, channel_conv.values,
              color=['#2ecc71' if v > df['converted'].mean() else '#e74c3c' for v in channel_conv.v
axes[0,0].axhline(df['converted'].mean(), color='navy', linestyle='--', label=f'[redacted] {df["convert
axes[0,0].set_title('[redacted]'); axes[0,0].set_ylabel('[redacted]')
axes[0,0].legend()
for i, (ch, val) in enumerate(channel_conv.items()):
    axes[0,0].text(i, val + 0.005, f'{val:.1%}', ha='center', fontsize=10)

# [redacted] vs [redacted]
device_conv = df.groupby('device')['converted'].mean().sort_values(ascending=False)
axes[0,1].bar(device_conv.index, device_conv.values, color=sns.color_palette('Set2', 3))
axes[0,1].axhline(df['converted'].mean(), color='navy', linestyle='--')
axes[0,1].set_title('[redacted]'); axes[0,1].set_ylabel('[redacted]')

# [redacted] vs [redacted] bin [redacted]
df['page_bin'] = pd.cut(df['browsing_pages'], bins=[0, 3, 6, 10, 15, 100],
                        labels=['1-3[redacted]', '4-6[redacted]', '7-10[redacted]', '11-15[redacted]', '15[redacted]'])
page_conv = df.groupby('page_bin', observed=True)['converted'].mean()
axes[1,0].bar(range(len(page_conv)), page_conv.values, color='steelblue', alpha=0.8)
axes[1,0].set_xticks(range(len(page_conv)))
axes[1,0].set_xticklabels(page_conv.index, rotation=20)
axes[1,0].set_title('[redacted] vs [redacted]')
axes[1,0].set_ylabel('[redacted]')

# [redacted] vs [redacted]
sns.boxplot(data=converted_df, x='city_tier', y='first_order_amount',
            order=['[redacted]', '[redacted]', '[redacted]'], palette='muted', ax=axes[1,1])
axes[1,1].set_title('[redacted] vs [redacted]')

plt.suptitle('[redacted] 3[redacted]', fontsize=14, y=1.01)
```

```
plt.tight_layout(); plt.show()

# ========== 步驟 4：產生假設 ==========
print("\n" + "=" * 50)
print("步驟 4：產生假設")
print("=" * 50)
hypotheses = [
    "H1：週末訂單量顯著高於平日，建議調整週末人力排班",
    "H2：新客戶在第一週的轉換率最低，建議加強首購引導流程",
    "H3：高客單價訂單集中在特定時段，建議推出時段限定優惠",
]
for h in hypotheses:
    print(f"  {h}")
print("\n[提示] M6: 把這些假設寫成可驗證的實驗")
```

視覺建議：
用四個步驟的標籤框架呈現整個流程，每個步驟顯示關鍵輸出（截圖或示意圖），並標注「在這個步驟發現了什

銜接下一張：「找到了洞察，怎麼讓別人也理解它？這需要把分析結果包裝成業務語言。」

Slide 10：如何用一頁圖表講業務故事

核心訊息：圖表本身不說話，說話的是圖表背後的詮釋結構：觀察 → 洞察 → 建議。

講師講解要點：

- 大多數分析師的圖表只停在「觀察」層：「這個月銷售額下降了 15%。」這是描述，不是洞察
- 觀察（Observation）：資料告訴我們什麼事實？客觀陳述，有數字支撐
- 範例：「桌機用戶的轉換率（32%）比手機用戶（18%）高出 78%」
- 洞察（Insight）：為什麼會這樣？背後的機制或模式是什麼？
- 範例：「桌機購物流程讓用戶能比較更多商品，複雜購物決策在桌機上完成率更高」
- 建議（Recommendation）：基於這個洞察，我們應該做什麼？
- 範例：「優先優化手機端的商品比較功能，目標是將手機轉換率提升至 25%」
- 一頁紙的力量：三段文字 + 一張主圖 + 一個洞察句標題，強迫自己聚焦

業務故事範本：

```
[redacted]
│
[redacted]
│
[redacted]
[redacted]
[redacted]
│
[redacted]
[redacted]
[redacted]
│
[redacted]
[redacted]
[redacted]
```

視覺建議：
示範一頁「業務故事圖」樣板，包含：大標題（洞察句，不是「銷售分析」），主圖（標注關鍵數據點），三段

銜接下一張：「理論說完了。現在輪到你，用真實資料做一次完整的 EDA。」

Slide 11：【練習 A】完整 EDA 探索

核心訊息：EDA 能力只能從實作中建立，不能從聽講中建立。

練習說明：

資料集：零售電商訂單資料（提供清洗後的 CSV，約 5000 筆）

欄位包含：訂單日期、用戶 ID、商品分類、購買金額、是否首購、城市等級、獲客渠道、瀏覽頁數

任務說明：

1. 看全貌（5 分鐘）
2. 執行 `df.info()`、`df.describe()`、缺值統計

記錄：資料維度、有問題的欄位、任何立即可見的異常

單變數分析（10 分鐘）

5. 至少對 3 個數值欄位畫 histogram 或 boxplot
6. 至少對 2 個類別欄位做 value_counts() 並畫 bar chart

記錄：哪些欄位分佈異常？有沒有異常值需要處理？

雙變數分析（10 分鐘）

9. 至少探索 3 組變數關係（包含數值×數值、數值×類別各至少一組）

10. 畫相關性矩陣熱力圖

記錄：哪些關係最強？有什麼出乎意料的發現？

形成假設（5 分鐘）

13. 列出 2-3 個「基於 EDA 發現，值得進一步驗證的業務假設」

評估標準： - 是否完整走過三個層次（單變數 → 雙變數） - 圖表選型是否符合問題類型 - 假設是否具體且可被驗證

時間：30 分鐘實作，5 分鐘小組分享

講師引導要點： - 提醒學員先寫問題再畫圖，不要打開資料就開始畫 - 引導觀察：「你畫這張圖想問什麼問題？這張圖有沒有回答到？」 - 常見卡關點：只有觀察沒有假設 → 追問「如果這個發現是真的，對業務意味著什麼？」

Slide 12：【練習 B】三段洞察簡報

核心訊息：分析的終點不是圖表，是可以讓他人理解並行動的洞察。

練習說明：

基於練習 A 的 EDA 結果，選出你認為最有業務價值的三個發現，製作三頁洞察簡報。

每一頁的格式：

```
## [illegible] #N [illegible]

### [illegible]
[illegible]

### [illegible]
[illegible]

### [illegible]
```

[redacted]

評估標準：

評估面向	優秀	及格	需改善
圖表選型	圖形類型完全匹配問題類型	圖形能回答問題但非最佳選擇	圖形和問題不匹配
觀察層	有具體數字，客觀陳述	有描述但缺乏數字支撐	只說「有差異」沒有程度
洞察層	有解釋機制，超越觀察	有部分解釋	洞察 = 觀察的重複
建議層	具體、可行動、有優先順序	方向正確但不夠具體	太模糊或不現實

時間：25 分鐘製作，10 分鐘互評分享

講師引導要點：
- 重點不在報告精美，在於洞察是否有「解釋力」
- 常見錯誤：把觀察層和洞察層寫成一樣的東西
- 追問技巧：「如果移除這張圖，你的結論還成立嗎？那就是說，這張圖沒有在支撐你的結論。」

補充：What -> So What -> Now What 洞察框架

這是顧問業與資料團隊最常用的洞察組織框架，能幫你把圖表上的發現，轉化為可以行動的商業建議。

層次	問的是	例子
What（觀察）	資料告訴我們什麼客觀事實？	「Q3 的退貨率從 5% 上升到 12%」
So What（洞察）	這代表什麼？為什麼重要？	「退貨集中在新品類，可能是品質控管或商品指

Now What（建議）	應該採取什麼行動？	「暫停新品類上架，先進行品質抽檢與商品頁改

使用時機： - 練習 B 的三段洞察簡報，每一頁都應該走完這三層 - EDA 報告的每個關鍵發現，都用這個框架組織 - 向非技術人員報告時，這三層就是你的故事結構

常見錯誤： - 只有 What 沒有 So What：「退貨率上升了」— 然後呢？ - So What 和 What 重複：「退貨率上升代表退貨變多了」— 這不是洞察 - Now What 太模糊：「應該關注退貨問題」— 要具體到可以行動

補充：視覺化反模式 — 常見錯誤與修正

反模式	問題	正確做法
用圓餅圖做比較	人眼對角度的判斷力遠不如長度	改用水平長條圖
Y 軸不從 0 開始	誇大差異，誤導讀者	折線圖可不從 0，長條圖必須從 0
3D 圖表	透視變形讓數值判讀失真	永遠用 2D，除非資料本身是三維
彩虹色（rainbow colormap）	色彩順序無直覺對應，色盲不友善	用連續漸層（viridis）或語義色（紅=負面、綠=
一張圖塞太多系列	超過 5-7 條線就無法區分	拆成多張子圖（subplot），或用互動式圖表
缺少標題和軸標籤	讀者不知道在看什麼	每張圖必須有標題，軸標籤含單位
裝飾性圖表	加了很多格線、陰影、背景色	「少即是多」：去除非必要的視覺元素（Tufte 原則）

一句話原則：如果移除一個視覺元素後圖表的訊息不變，那就移除它。

模組總結

本模組的核心主線

視覺化工具（Matplotlib + Seaborn）只是手段，EDA 方法論才是核心。工具知道怎麼用，但只有掌握「單變數 -> 雙變數 -> 多變數」的系統路徑，才能從資料中提取有價值的洞察。

四個不能忘的原則

1. 先問問題，再畫圖：每一張圖都應該對應一個具體問題
2. 異常值先判斷再處理：刪除之前，先問「這是錯誤還是訊號？」
3. 洞察必須超越觀察：圖表不說話，詮釋才說話（用 What -> So What -> Now What 框架）
4. 避開反模式：圓餅圖少用、3D 不用、彩虹色不用、標題軸標必寫

下一步：M6 統計思維與假設檢定

EDA 的終點是形成假設。M6 將學習如何用統計工具驗證這些假設——把「看起來有差異」轉化為「在 X% 信心水準下確認有差異」。本模組在 EDA 中發現的那些「可能的關係」，在 M6 會學到怎麼嚴謹地確認它們。

參考文獻與引用來源

經典文獻

1. Tukey, J.W. (1977). Exploratory Data Analysis. Pearson. — EDA 方法論的奠基之作。「The greatest value of a picture is when it forces us to notice what we never expected to see.」出自本書第 vi 頁。
2. Anscombe, F.J. (1973). "Graphs in Statistical Analysis." The American Statistician, 27(1), 17-21. — Anscombe's Quartet

的原始論文，示範統計量相同但分布截然不同的四組資料。doi.org/10.1080/00031305.1973.10478966

視覺化原則

1. Tufte, E.R. (2001). The Visual Display of Quantitative Information, 2nd Edition. Graphics Press. — 資料視覺化設計的聖經，「data-ink ratio」原則的來源。
2. Cairo, A. (2016). The Truthful Art: Data, Charts, and Maps for Communication. New Riders. — 資料新聞與商業視覺化的實務指南。
3. Knaflic, C.N. (2015). Storytelling with Data. Wiley. — 「用資料說故事」的商業簡報方法論。

洞察框架

1. What -> So What -> Now What 框架 — 此框架廣泛使用於管理顧問業（McKinsey、BCG 等）與企業資料團隊。最早的系統化描述可追溯至 Minto, B. (1987). The Pyramid Principle. Pearson. 該書提出的金字塔原理強調「結論先行、以上統下」的思維方式。

視覺化反模式

1. 圓餅圖的判讀問題 — Cleveland, W.S. & McGill, R. (1984). "Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods." Journal of the American Statistical Association, 79(387), 531-554.
此研究證實人眼對長度的判讀精度遠高於角度。
2. 色彩選擇 — viridis 色票的設計考量可參見 matplotlib.org/stable/users/explain/colors/colormaps.html

工具文件

1. Matplotlib Official Documentation — matplotlib.org
2. Seaborn Official Documentation — seaborn.pydata.org

延伸閱讀

- Tukey, J.W. (1977). Exploratory Data Analysis — EDA 的奠基之作，確立了「探索」而非「確認」的分析哲學
- Cairo, A. (2016). The Truthful Art — 視覺化的倫理與設計原則，每位資料分析師必讀
- Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment — Matplotlib 作者的原始論文，理解設計哲學
- Waskom, M. (2021). Seaborn: Statistical Data Visualization — Seaborn 官方文件，包含所有圖形的詳細參數說明
- Wickham, H. (2010). A Layered Grammar of Graphics — 圖形語法的理論基礎，理解「為什麼選這個圖」